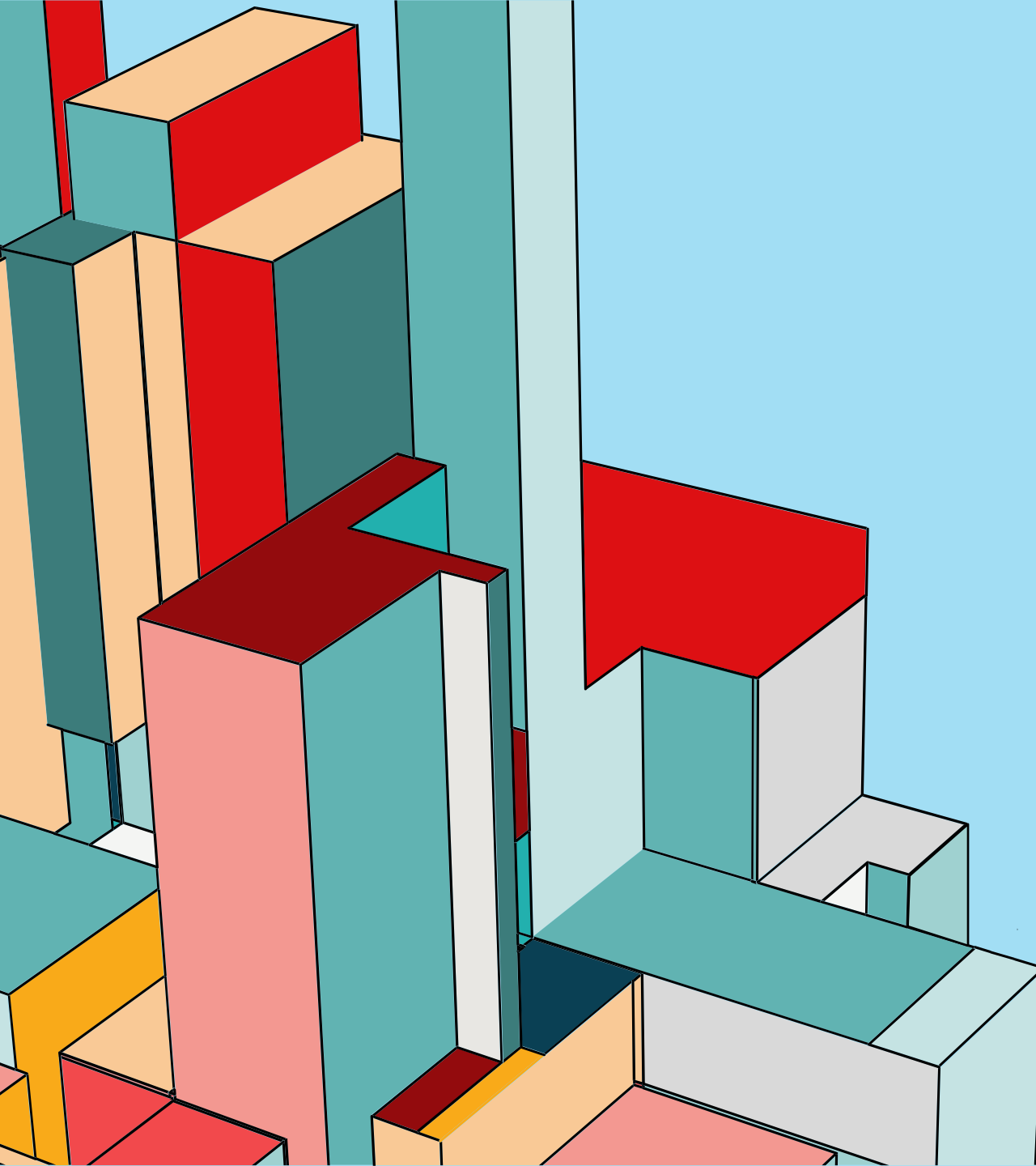




LECTURE 24 **NP-COMPLETE PROBLEMS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

P VS NP

Recall from our brief discussion of greedy approximation algorithms that computer scientists are very concerned with two broad classes of problems.

- P is the set of problems that can be solved in polynomial time or faster.
- NP is the set of problems whose solutions can be verified in polynomial time or faster. (Alternate def'n: the set of problems that can be solved by a nondeterministic machine in poly time or faster)

An obvious consequence of these definitions is that $P \subseteq NP$.

Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Log-linear	$O(n \log n)$
Polynomial	$O(n^c)$
Quasi-polynomial	$O(2^{(\log n)^c})$
Subexponential	$O(2^{o(n)})$
Exponential	$O(2^n)$
Factorial	$O(n!), O(n^n)$
Worse	$O(2^{2^n}), \text{ etc.}$

DEFINE THE WORD “VERIFY”...

Let us be precise about what we mean by “verify”.

For now we will limit ourselves to *decision problems* – ones where the answer is simply True or False.

Let $\mathfrak{D}$ be a decision problem, and let x be a string containing a valid input for $\mathfrak{D}$. Further, let y be a proposed solution.

For example, $\mathfrak{D}(x)$ might be “ x has a network flow of volume at least 20.” and then x would contain a description of a valid flow network. y would then be some actual flow that supposedly has a volume ≥ 20 .

Technical note: We restrict ourselves to cases where the length of y is at most a polynomial function of x . If it could be longer, there would be no way that $\mathfrak{D}$ could possibly be a member of P or NP .

DEFINE THE WORD “VERIFY”...

A verifier $\mathcal{V}_{\mathfrak{D}}$ is then an algorithm that takes x and y (the problem input and the proposed solution) as inputs and returns **True** whenever y describes a situation that forces the answer to $\mathfrak{D}(x)$ to be True.

In some sense, $\mathcal{V}_{\mathfrak{D}}$ must be “convinced” by a reasonably short “proof” (in the form of y) that $\mathfrak{D}(x)$ is True.

If $\mathcal{V}_{\mathfrak{D}}$ is also a polynomial-time algorithm, then $\mathfrak{D} \in NP$.

Note that any correct polynomial-time verifier $\mathcal{V}_{\mathfrak{D}}$ guarantees that $\mathfrak{D}$ has a solution that runs in no worse than exponential time: brute force guess-and-check.

USING REDUCTIONS TO TALK ABOUT DIFFICULTY

When we talk about P and NP , we talk about how “easy” or “hard” a given problem might be. It would be nice to formalize our notion of difficulty.

The tool that we use for this is *reduction* – the process of taking an instance of one problem and turning into a different problem.

The question to ask is: “If you had a solver for problem $\mathfrak{D}$, could you use it to solve every instance of problem $\mathfrak{C}$?”

```
def solveC(inputC):  
  
    inputD = constructed  
             based on inputC  
  
    answerD = solved(inputD)  
    # solved is a “black  
    # box” that we can call  
  
    answerC = constructed  
             based on answerD  
  
    return answerC
```

USING REDUCTIONS TO TALK ABOUT DIFFICULTY

If we can do this, we write that problem $\mathfrak{C}$ *reduces to* problem $\mathfrak{D}$, or just $\mathfrak{C} \leq \mathfrak{D}$.

This is a statement about the relative difficulty of $\mathfrak{C}$ and $\mathfrak{D}$: Problem $\mathfrak{C}$ is no harder than problem $\mathfrak{D}$.

Important: It does not guarantee that $\mathfrak{D}$ is more difficult – they could be equivalent.

The prepositions can throw people here: “ $\mathfrak{C}$ reduces to $\mathfrak{D}$ ” means that $\mathfrak{D}$ is the (possibly) harder side of the reduction. We also sometimes write about a reduction from $\mathfrak{C}$; “from” indicates the (possibly) easier side.

```
def solveC(inputC):  
    inputD = constructed  
             based on inputC  
  
    answerD = solved(inputD)  
    # solved is a “black  
    # box” that we can call  
  
    answerC = constructed  
             based on answerD  
  
    return answerC
```

EXAMPLE REDUCTIONS

Problem $\mathfrak{C}$ = Find the median of a list

Problem $\mathfrak{D}$ = Sort a list of numbers

- Input transformation: None needed!
- Output transformation: Select the element at index $n/2$.

Thus, $\mathfrak{C} \leq \mathfrak{D}$.

```
def solveC(inputC):
```

```
    inputD = constructed  
             based on inputC
```

```
    answerD = solveD(inputD)  
    # solveD is a “black  
    # box” that we can call
```

```
    answerC = constructed  
             based on answerD
```

```
    return answerC
```

EXAMPLE REDUCTIONS

Problem $\mathfrak{C}$ = Maximum bipartite matching

Problem $\mathfrak{D}$ = Maximum network flow

- Input transformation: Connect source to all nodes on one side and all nodes on other side to sink. Set all capacities to 1.
- Output transformation: Select the edges that have flow.

Thus, $\mathfrak{C} \leq \mathfrak{D}$.

```
def solveC(inputC):
```

```
    inputD = constructed  
            based on inputC
```

```
    answerD = solveD(inputD)  
    # solveD is a “black  
    # box” that we can call
```

```
    answerC = constructed  
            based on answerD
```

```
    return answerC
```

THE SPEED OF REDUCTION

If all we care about is *decidability* (i.e. “can problem $\mathcal{C}$ be solved at all?”), this is sufficient.

But P and NP care depend upon the number of operations being polynomial in the size of the input.

And so, we need to discuss how much work is needed to transform the input and to transform the answer.

```
def solveC(inputC):  
  
    inputD = constructed  
            based on inputC  
  
    answerD = solved(inputD)  
    # solved is a “black  
    # box” that we can call  
  
    answerC = constructed  
              based on answerD  
  
    return answerC
```

THE SPEED OF REDUCTION: $\leq_P$

If our reduction is one in which the **input transformation** and **output transformation** can each be done in polynomial or better time, then we write that $\mathfrak{C} \leq_P \mathfrak{D}$.

This is called *polynomial-time reducibility* (or sometimes *many-one reducibility*).

Technically, we can also call our **black-box solver for $\mathfrak{D}$** a polynomial number of times as well. In practice, we nearly always just need the one call.

```
def solveC(inputC):  
  
    inputD = constructed  
             based on inputC  
  
    answerD = solveD(inputD)  
    # solveD is a “black  
    # box” that we can call  
  
    answerC = constructed  
               based on answerD  
  
    return answerC
```

REDUCTION, P, & NP

Suppose we have shown that $\mathfrak{C} \leq_P \mathfrak{D}$.
Which of the following are true?

- A) If $\mathfrak{C} \in P$, then $\mathfrak{D} \in P$.
- B) If $\mathfrak{C} \notin P$, then $\mathfrak{D} \notin P$.
- C) If $\mathfrak{D} \in P$, then $\mathfrak{C} \in P$.
- D) If $\mathfrak{D} \notin P$, then $\mathfrak{C} \notin P$.

Answer now in the Submittity poll.



TWO WAYS TO USE REDUCTION

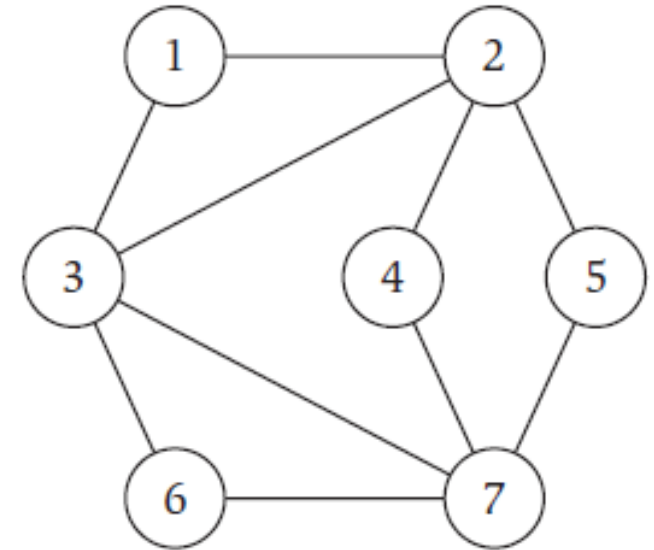
We have used reduction a number of times in this course already.

All of the previous instances have been targeted at "easy" problems: We have a fast solver for $\mathfrak{D}$, and then we show $\mathfrak{C} \leq_P \mathfrak{D}$, which immediately gives us a fast solver for $\mathfrak{C}$. ($\mathfrak{D} \in P \Rightarrow \mathfrak{C} \in P$).

In our next couple of classes, we'll be using reduction in the other direction: We know (or at least suspect) that $\mathfrak{C}$ cannot be solved quickly. Then when we show $\mathfrak{C} \leq_P \mathfrak{D}$, we conclude that $\mathfrak{D}$ also cannot be solved quickly. ($\mathfrak{C} \notin P \Rightarrow \mathfrak{D} \notin P$)

THE INDEPENDENT SET PROBLEM

Given a graph G , find the largest $S \subseteq V$ such that no two vertices in S are neighbors.



QUICK ASIDE: OPTIMIZATION PROBLEMS VS DECISION PROBLEMS

Many many problems in CS are *optimization problems*: They ask us to minimize or maximize some quantity, given some constraints on the solution.

For each such optimization problem $\mathcal{O}$, there is a related decision problem $\mathcal{D}$ that asks: "Is there a solution that with value at least/at most k ?"

Obviously, $\mathcal{D} \leq_P \mathcal{O}$ – it only takes $O(1)$ additional work to check k against the calculated optimum value.

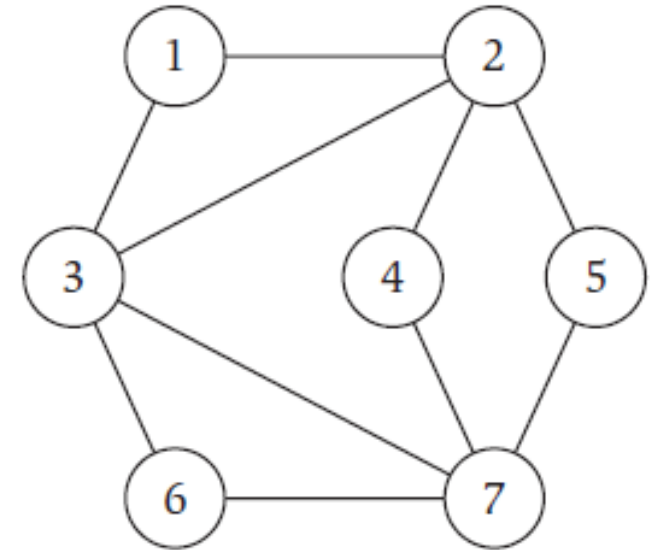
Less obviously, $\mathcal{O} \leq_P \mathcal{D}$ – we can try many different values of k , and use binary search to home in on the optimum value. This requires $O(\log k)$ trials. We can thus write $\mathcal{O} \equiv_P \mathcal{D}$.

THE INDEPENDENT SET PROBLEM

Given a graph G , find the largest $S \subseteq V$ such that no two vertices in S are neighbors.

Given a graph G , is there a subset $S \subseteq V$ where $|S| = k$ such that no two vertices within S are neighbors?

- When $k = 7$?
- When $k = 3$?
- When $k = 5$?
- When $k = 4$?

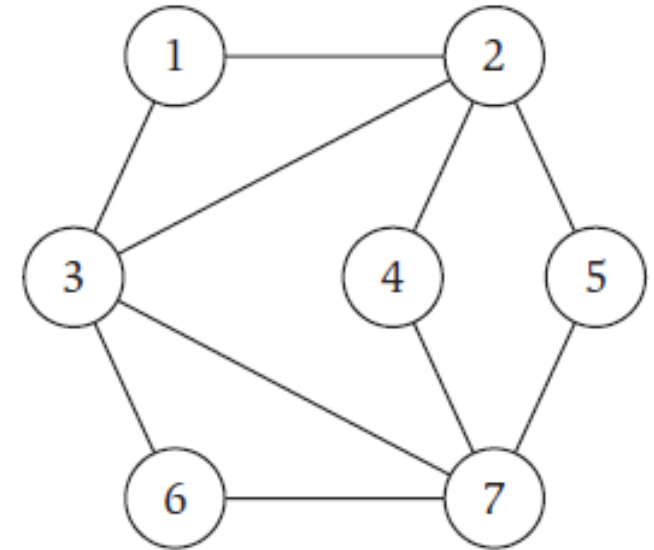


THE VERTEX COVER PROBLEM

Given a graph G , is there a subset $S \subseteq V$ where $|S| = k$ such that every edge touches one of the vertices in S ?

Which problem is harder: Independent Set or Vertex Cover?

Answer: Neither!

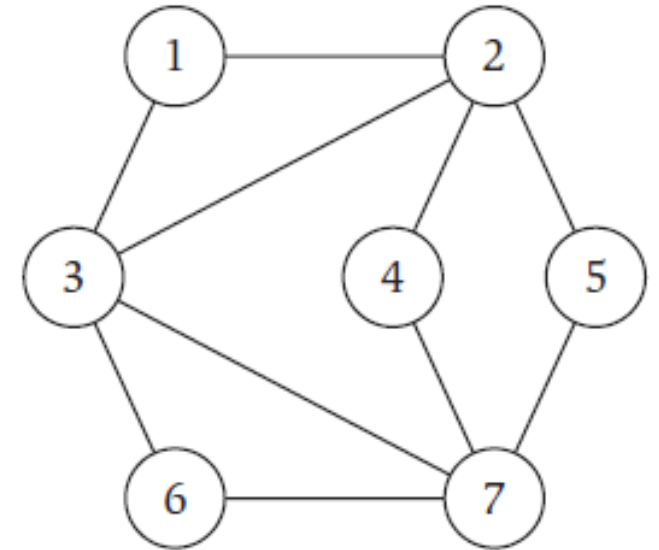


PROOF OF EQUIVALENCE

Claim: In any graph G , $S \subseteq V$ is a vertex cover if and only if $V - S$ is an independent set.

Proof (if): Assume S is a vertex cover. Consider any two nodes in $V - S$. If there were an edge between them, then there is an edge not covered by S . ■

Proof (only if): Assume $V - S$ is an independent set. Consider an arbitrary edge. It cannot have both ends in $V - S$; therefore, at least one end must be in S . Thus, S meets the definition of a vertex cover. ■



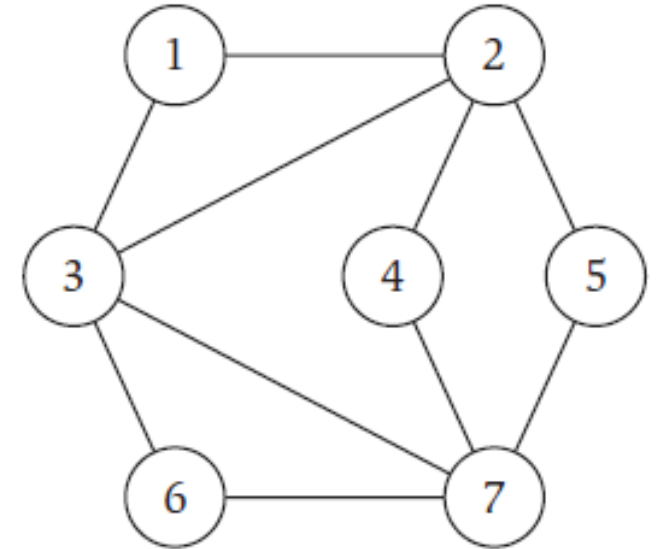
REDUCTIONS

Independent Set $\leq_P$ Vertex Cover

Given a black-box solver for VC, we can find an IS of size k by asking the black box for a VC of size $|V| - k$ and then taking the complement of the output. (Complement is an $O(n)$ operation.)

Vertex Cover $\leq_P$ Independent Set

Given a black-box solver for IS, we can find a VC of size k by asking the black box for an IS of size $|V| - k$ and then taking the complement of the output. (Complement is an $O(n)$ operation.)

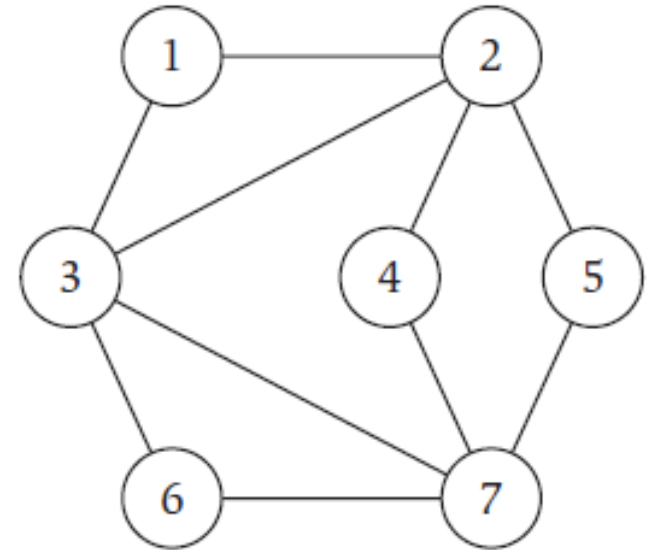


EQUIVALENT PROBLEMS... SO WHAT DO WE KNOW?

$IS \equiv_P VC$, which means that if we find a good solution for one of them, we will also have a good solution for the other.

Observe that we can check any proposed solution quickly. IS and VC are both in *NP*.

However, we don't yet have any fast ways to produce solutions – we don't know if these problems are in the class *P*.



THE HARDEST PROBLEMS

Imagine if we could find a problem $\mathcal{S}$ such that we could prove these two facts about $\mathcal{S}$:

- $\mathcal{S} \in NP$
- $\forall \mathcal{R} \in NP, \mathcal{R} \leq_p \mathcal{S}$ (!!!)

The notion seems ludicrous – that second condition is crazy.

But – if we could find such a problem $\mathcal{S}$, it would be, in a very real sense, the hardest possible problem in NP .

We even have a term for such problems: NP -complete.

THE SKELETON KEY PROBLEM: SAT

One of the most generic problems in computer science is the *Boolean satisfiability problem* (or *SAT*). It asks: "Given an expression in Boolean logic, is it possible to assign values to the variables such that it comes out to True?"

For example:

- $(a \wedge b) \vee \neg(b \wedge \neg c)$ - sure, let $a = T, b = T, c = \text{any}$
- $(a \vee b) \wedge (\neg a \vee \neg b)$ - yup, we can use $a = T, b = F$
- $(a \vee b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg b)$ - can't do it!

STANDARDIZED SAT

To make it a bit easier to work with, we can put all of our Boolean expressions into a standard form.

The most common one is CNF – *conjunctive normal form*.

- We group terms (single variables or their negations) into clauses (terms joined by OR); the clauses are then all ANDed together.
- DeMorgan's Laws guarantee that we can always put a Boolean expression into CNF.
- With a bit more work, we can force every clauses to contain exactly three terms; this version of the problem is known as 3-SAT.

THE COOK-LEVIN THEOREM

At the very first ACM Symposium on the Theory of Computation (STAC) in 1971, Stephen Cook published "The Complexity of Theorem-Proving Procedures".

Here are the first couple of sentences of the abstract:

It is shown that **any** recognition problem solved by a **polynomial time-bounded nondeterministic Turing machine** can be "reduced" to the problem of determining whether a given propositional formula is a tautology. Here "reduced" means, roughly speaking, that the first problem can be solved deterministically in polynomial time provided an oracle is available for solving the second.

In case you didn't clock it: Cook is saying that SAT is *NP*-complete!

THE COOK-LEVIN THEOREM: SAT IS NP-COMPLETE

Leonid Levin in the USSR independently proved the same idea at roughly the same time, but it was really [Richard Karp](#) who took the ball and ran with it. His proof is outlined here:

Any problem $\mathfrak{D} \in NP$ has, by definition, a nondeterministic Turing Machine $\mathcal{M}_{\mathfrak{D}}$ that can produce a solution in polynomial time. Call the polynomial in question $p(n)$.

Every individual operation of $\mathcal{M}_{\mathfrak{D}}$, plus $\mathcal{M}_{\mathfrak{D}}$ itself, can be encoded in a giant Boolean expression. (Details omitted, but ultimately, everything in our computers is done with AND, OR, and NOT.) The encoding used requires $O\left((p(n))^3 \cdot \log(p(n))\right)$ time to create, and it has a satisfying assignment precisely when $\mathcal{M}_{\mathfrak{D}}$ returns TRUE.

Thus, $\forall \mathfrak{D} \in NP, \mathfrak{D} \leq_P \text{SAT}$. (And so also $\mathfrak{D} \leq_P \text{3-SAT}$.) ■

BLOWING THE DOORS WIDE OPEN

With one NP -complete problem $\mathcal{S}$ in hand, all it takes to find another is to show that $\mathcal{S}$ is polynomially reducible to some other problem $\mathcal{T} \in NP$.

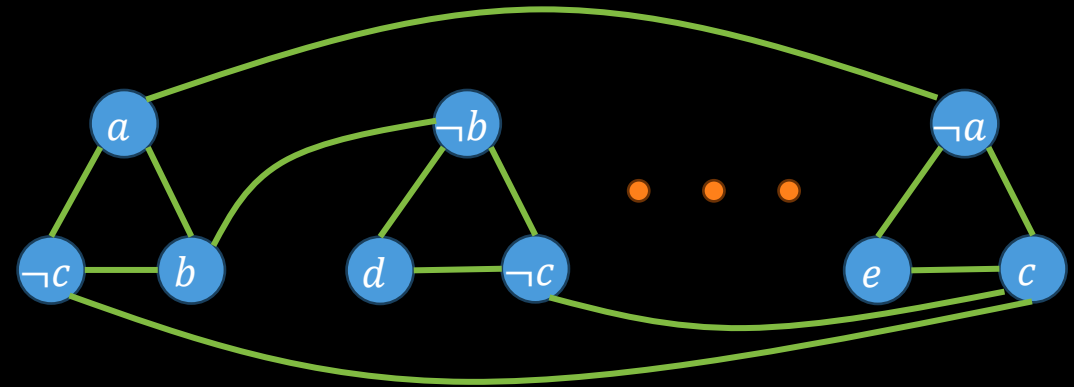
...

3-SAT $\leq_P$ Independent Set

INDEPENDENT SET IS *NP*-COMPLETE

- To show $3\text{-SAT} \leq_p \text{IS}$, we need to provide a transformation from an instance of 3-SAT to an instance of IS such that the IS solution can be transformed back into a 3-SAT solution.
- Claim: $\text{IS}(k)$ has a solution on the constructed graph iff the 3-SAT instance is satisfiable.

$(a \vee b \vee \neg c) \wedge (\neg b \vee \neg c \vee d) \wedge \dots$
 $\wedge (\neg a \vee c \vee e)$ (k clauses overall)

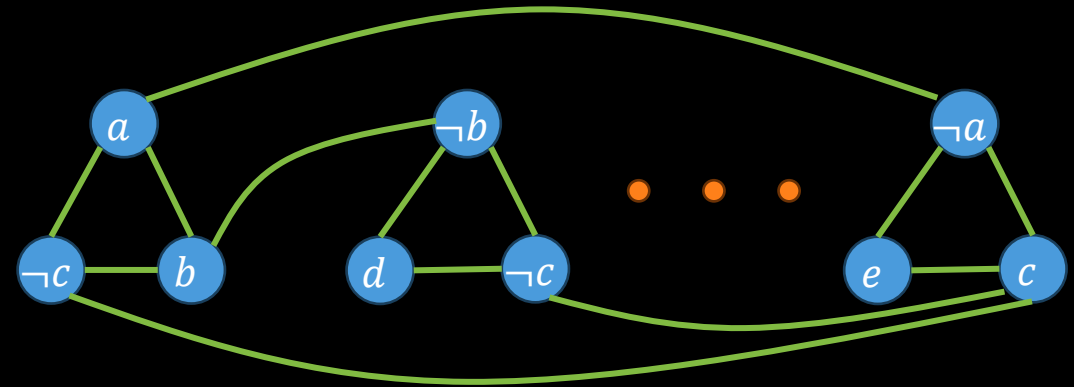


Make a triangle for every clause,
and then connect opposites.

INDEPENDENT SET IS *NP*-COMPLETE

- If the 3-SAT instance is satisfiable, then every clause has a term that is true. The vertices corresponding to those terms will form an independent set.
- If the graph has an independent set of size k , then it must have precisely one node from each triangle. For each Boolean variable b_i , if a node corresponding to b_i is in the IS, let $b_i = 1$. If a $\neg b_i$ node is in the IS, let $b_i = 0$. (If neither, choose freely.)

$(a \vee b \vee \neg c) \wedge (\neg b \vee \neg c \vee d) \wedge \dots$
 $\wedge (\neg a \vee c \vee e)$ (k clauses overall)



Make a triangle for every clause,
and then connect opposites.

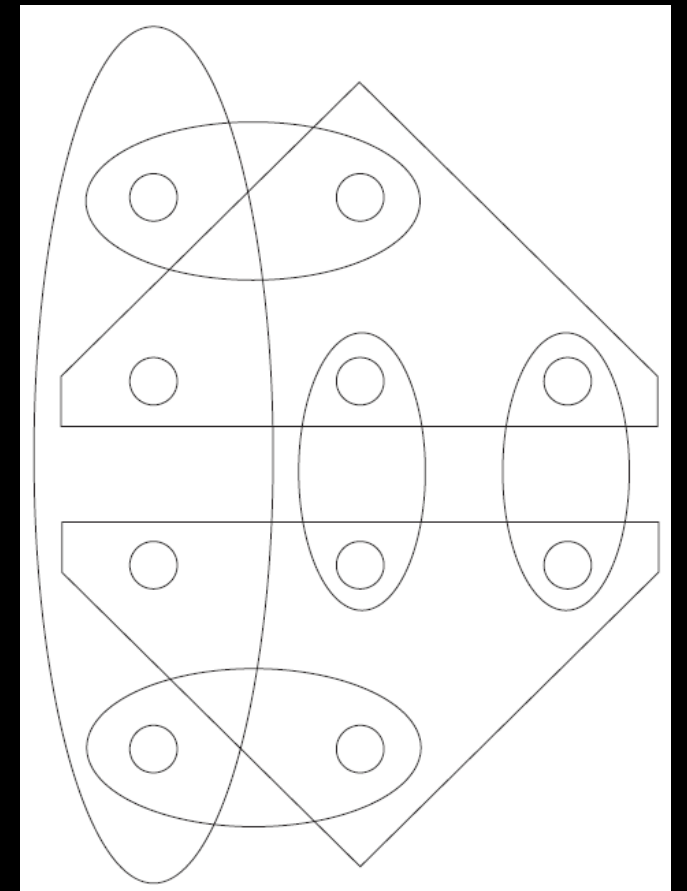
Immediately, vertex cover is *NP*-
complete as well (by transitivity).

GENERAL STRATEGY FOR NP-COMPLETE REDUCTIONS

- Given some problem X , to prove that it is NP -complete:
 - Prove that $X \in NP$.
 - Choose a known NP -complete problem Y .
 - Provide a polynomial-time transformation of an arbitrary instance s_Y of problem Y into an instance s_X of problem X , such that:
 - Whenever s_Y returns TRUE, s_X also returns TRUE. **AND**
 - Whenever s_X returns TRUE, s_Y also returns TRUE.
- One easy possibility: If you can show that a problem is a more general version of a known NP -complete problem, you're already done.

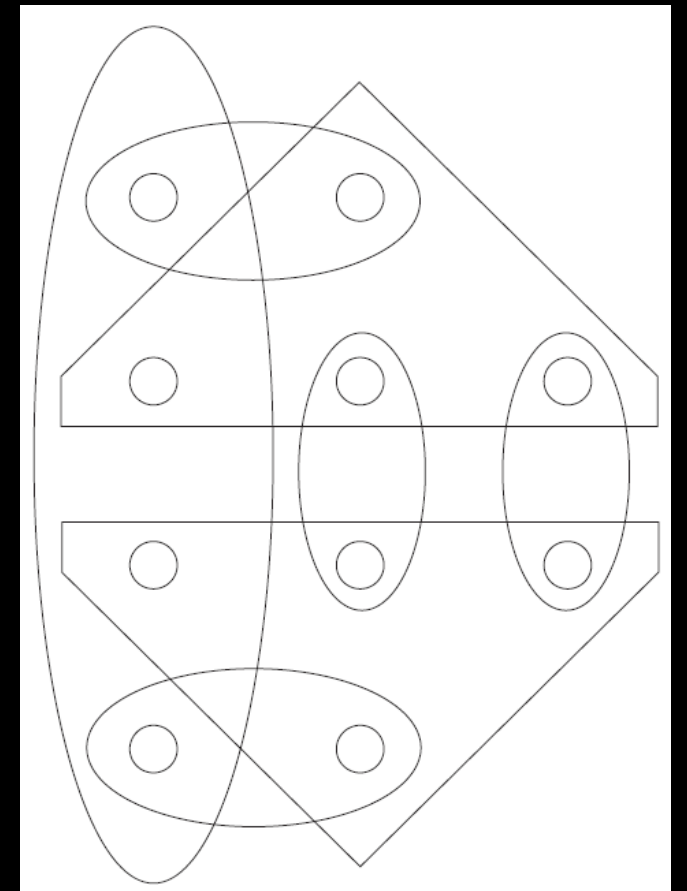
THE SET COVER PROBLEM

- Given some collection of objects U and a series of subsets of those objects $S_1, S_2, \dots, S_n \subseteq U$, find k of the S_i whose union is U .
 - “Cover” the entire universal set via a collection of k subsets. (minimize k)
- Note that Vertex Cover is a special case of Set Cover.
 - U is the set of edges. Each vertex v defines a subset S_v of edges that it touches.



THE SET COVER PROBLEM

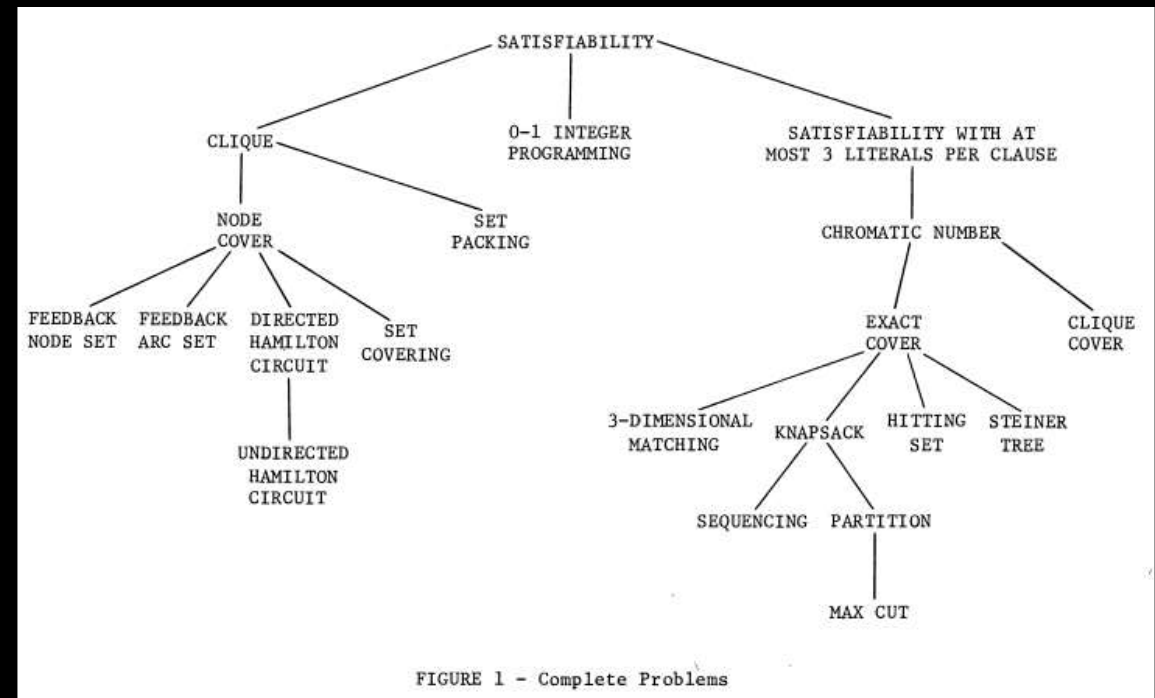
- Since we can map VC onto SC with essentially no effort, $VC \leq_p SC$.
- But we've already shown that VC is *NP*-complete, so...
- ... Set Cover is also *NP*-complete.

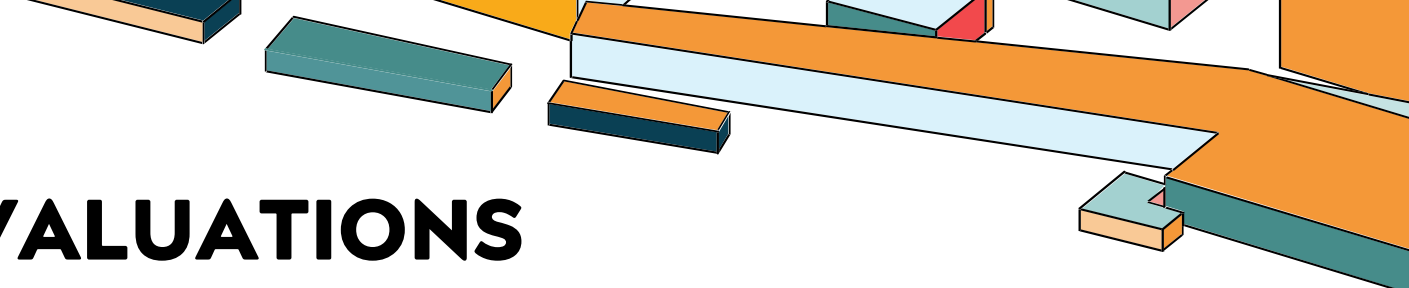


ENTER RICHARD KARP...

- In 1972, Karp presented “Reducibility Among Combinatorial Problems” at IBM’s symposium on Complexity of Computer Computations at Watson Labs in Yorktown Heights.

“We show that a large number of classic unsolved problems of covering, matching, packing, routing, assignment and sequencing are equivalent, in the sense that either each of them possesses a polynomial-bounded algorithm or none of them does.”





INSTITUTE COURSE EVALUATIONS

- RPI's formal course evaluations ARE NOW OPEN. They will be available through 4/30.
- The department, school, and Institute take these seriously.
- The accuracy of the data depends upon a high participation rate.
- Thus, I will incentivize completion as follows:
 - There are 225 students in the course. 75% is around 170.
 - Each student beyond 170 that completes a course evaluation will result in everyone getting a half-point bonus on their final exam (which will be out of 200 points total).

SECOND POLL

Make sure to submit an answer before leaving.

